

Stochastic Modeling Final Project

Gytis Vėjelis

2025

Task

In 1978, an influenza epidemic (caused by the H1N1 virus) broke out in a boarding school in Great Britain, where 763 pupils studied and lived. The epidemic affected more than two-thirds of all the pupils (see Fig. 3). It began when the students returned to school after the holidays—one of them arrived already ill. To stop the epidemic, each pupil who developed a fever was immediately isolated from others and confined to bed as soon as the first flu symptoms appeared. After a few days, their condition improved, and after several more days they fully recovered.

The table below shows how, during the epidemic, the number of pupils who were ill (confined to bed) and those who were recovering (convalescent) changed over time.

Table 1: Amount of ill and recovering pupils during the epidemic.

Day of epidemic (counting from January 22)	1	2	3	4	5	6	7	8	9	10	11	12	13
Number of ill pupils	3	8	26	76	225	298	258	233	189	128	68	29	14
Number of recovering pupils	0	0	0	0	9	17	105	162	176	166	150	85	47

- Propose a model that is consistent with the statistical data presented in Table 4.
- Perform numerical simulation and show that, with appropriate adjustment of the model parameters, your model can indeed describe this data.
- Based on your model, determine:
 - How many pupils in total became ill (compare this calculated number with the actual value of 512)?
 - On average, how much time passed between a pupil's infection and the appearance of the first symptoms of the disease?
 - On average, how many days did an ill pupil spend confined to bed, and after how many additional days of improvement did they fully recover?
 - On which day of the epidemic did the total number of infected pupils (both those newly infected but not yet showing symptoms and those already confined to bed) reach its peak, and what was this number?
- Analyze how vaccination could help prevent the epidemic. Calculate what fraction of unvaccinated pupils would become infected if $n\%$ of all pupils (for example, in the range from 10% to 90%) had been vaccinated before returning to school.

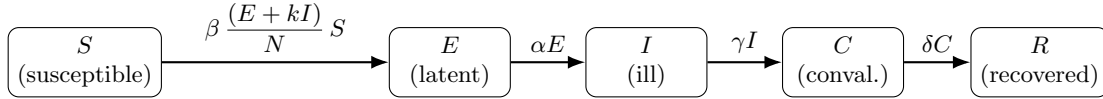
Introduction

In 1947 John von Neumann, remarked *"truth is much too complicated to allow anything but approximations"*. Epidemic models reflect this idea with a wide variety of possible models. Some are much too simple like a birth/death model, while others, require machine learning algorithms, consequently large datasets and thus are too complicated for such a task. Instead a stochastic SEICR model is chosen for this problem. This model divides the population into five possible categories:

1. (S) Susceptible - population which is susceptible to the disease, in this case all of pupils
2. (E) Exposed - these are the people who are exposed to the disease, we include this category since many epidemics have a latency period
3. (I) Infectious - people who are infected and are able to infect others
4. (C) Convalescent - population which is ill, but recovering
5. (R) Recovered - Final category of recovered people

Mathematical Modeling

Given these definitions is it then clear how we can create a continuous time markov chain which connects all these states:



First thing to notice is that the total number of pupils is conserved

$$N = S + E + I + C + R = \text{const} \quad (1)$$

with S , E , I , C and R are number of pupils that are susceptible, exposed, infectious, convalescent and recovered accordingly. Then the state space is just

$$\mathcal{S} = \{(s, e, i, c, r) \in \mathbb{N}^5 | s + e + i + c + r = N\}$$

and the state is simply a vector

$$X(t) = (S(t), E(t), I(t), C(t), R(t))$$

We know that the evolution of this state obeys the Markov property. The evolution of this state can be seen easily derived from the graphic above

$$\begin{aligned} \frac{dS}{dt} &= -\lambda(t)S; & \frac{dE}{dt} &= \lambda(t)S - \alpha E; & \frac{dI}{dt} &= \alpha E - \gamma I; \\ \frac{dC}{dt} &= \gamma I - \delta C; & \frac{dR}{dt} &= \delta C. \end{aligned} \quad (2)$$

where we use a force of infection function $\lambda(t)$

$$\lambda(t) = \beta \frac{E(t) + kI(t)}{N} \quad k \in [0, 1)$$

From these equations we can read off the rates

- $S \rightarrow E \quad a_1 = \beta \frac{E+I}{N} S$
- $E \rightarrow I \quad a_2 = \alpha E$
- $I \rightarrow C \quad a_3 = \gamma I$
- $C \rightarrow R \quad a_4 = \delta C$

with the total event rate being $a_0 = a_1 + a_2 + a_3 + a_4$. This allows to compute probabilities of certain events happening

$$P(\text{j event}) = \frac{a_j}{a_0} \quad (3)$$

Modeling

The modeling is done in two main steps. First we need to find out the parameter values for $\alpha, \beta, \gamma, \delta$ and then we can use Monte Carlo simulations with Gillespie algorithm. The parameter values are obtained by solving the set of differential equations (2) and optimising the solutions to fit the given data with python's `scipy.optimize` library. We use least squares optimisation method which computes the error as follows

$$\text{error} = \sum_t (I_{\text{model}}(t) - I_{\text{obs}})^2 + \sum_t (C_{\text{model}}(t) - C_{\text{obs}}(t))^2$$

where the indices "model" and "obs" are the models and observed quantities correspondingly. Thus we obtain the following values

```
1 beta = 5.8866
2 alpha = 3.1004
3 gamma = 0.4983
4 delta = 0.5948
```

To capture the stochastic nature of the model we use the aforementioned monte carlo simulation with the Gillespie algorithm. The algorithm works in the following way:

1. Initialise the variables
2. Loop while $t < T$
 - Compute event rates $()$ and the total rate a_0
 - Get random waiting time with $\delta t \sim \exp(a_0)$
 - Update time $t \leftarrow t + \delta t$
 - Pick random event using probabilities (3)
 - Update the state
3. Return the recorded array of states

Each simulation produces one possible epidemic realization, and by running many such simulations we can then compute the mean values.

- (a) Total ever became ill (stochastic mean) = 543.0 pupils
- (b) Mean infection to symptoms = $1/\alpha = 0.323$ days

- (c) Mean days ill in bed = $1/\gamma = 2.007$ days
 Mean extra convalescent = $1/\delta = 1.681$ days
- (d) Peak of (E+I) deterministic: day 6.60, size 363.9

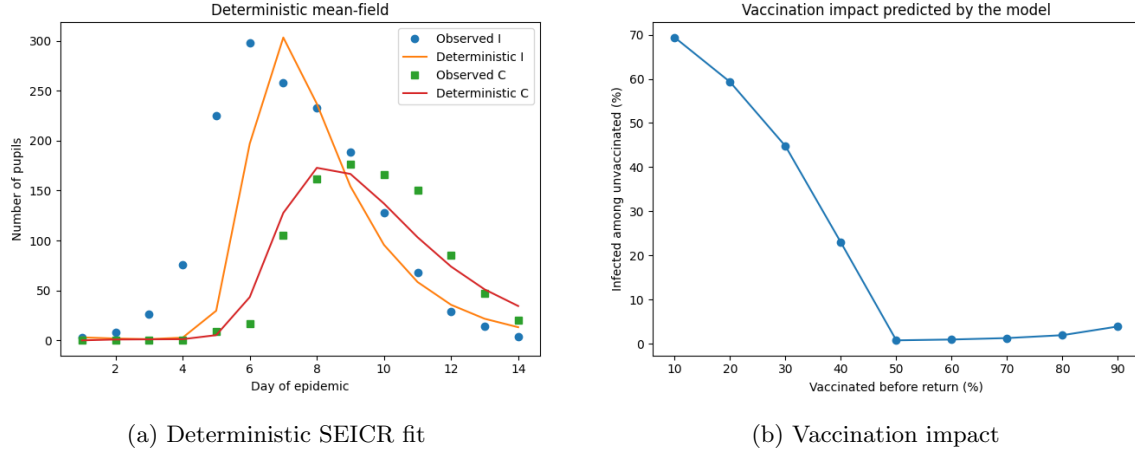


Figure 1: SEICR epidemic model results for pupil outbreak

We can also plot the results. Figure 1 (a) shows how the deterministic SEICR model captured the outbreak shape. In general the model reproduces the order of magnitude and general evolution ill (I) and Convalescent (C) pupils. However, the peak is shifted and the early growth of infections is later than the observed data. Much of the discrepancy can be attributed to the simplified model used: constant parameters, imperfect knowledge of initial conditions (especially initial latent infections) and most importantly the fact that the deterministic model reflects the *average* behaviour, while the true outbreak is a stochastic process. Figure 2 (b) shows the vaccination impact curve which has a non trivial tendencies. For vaccination levels under 30% – 40% the majority of the population still becomes infected. After increasing the number of vaccinated pupils to about 50% of pupils, the model shows a sharp drop in outbreak size. Beyond this threshold, a herd immunity has formed and further vaccination provides negligible results.

Appendix: Code

This appendix include the code written for this project

Script which solves and optimises ODE's

```

1 days = np.arange(1, 15)
2 I_obs = np.array([3, 8, 26, 76, 225, 298, 258, 233, 189, 128, 68, 29, 14, 4], dtype=
    float)
3 C_obs = np.array([0, 0, 0, 0, 9, 17, 105, 162, 176, 166, 150, 85, 47, 20], dtype=
    float)
4 N = 763.0
5
6 t_obs = np.arange(0, 14, dtype=float)
7
8 def seicr_rhs(t, y, beta, k, alpha, gamma, delta):
9     S, E, I, C, R = y
10    lam = beta * (E + k * I) / N
11    dS = -lam * S
12    dE = lam * S - alpha * E
13    dI = alpha * E - gamma * I
14    dC = gamma * I - delta * C
15    dR = delta * C
16    return [dS, dE, dI, dC, dR]
17
18 k_fixed = 0.0001
19
20 def simulate_ode(beta, alpha, gamma, delta):
21     I0 = float(I_obs[0]) # day1 ill
22     E0 = 0
23     C0 = 0.0
24     R0 = 0.0
25     S0 = N - I0
26     y0 = [S0, E0, I0, C0, R0]
27     sol = solve_ivp(
28         seicr_rhs, (0, 13), y0,
29         t_eval=t_obs,
30         args=(beta, k_fixed, alpha, gamma, delta),
31         rtol=1e-8, atol=1e-10
32     )
33     return sol.y
34
35 def residuals(theta):
36     beta, alpha, gamma, delta = np.exp(theta)
37     Y = simulate_ode(beta, alpha, gamma, delta)
38     I_hat, C_hat = Y[2], Y[3]
39
40     rI = (I_hat - I_obs) / np.sqrt(I_obs + 1.0)
41     rC = (C_hat - C_obs) / np.sqrt(C_obs + 1.0)
42     return np.concatenate([rI, rC])
43
44 theta0 = np.log([1.0, 1/1.2, 1/3.0, 1/3.0])
45 res = least_squares(residuals, theta0, max_nfev=8000)
46 beta, alpha, gamma, delta = np.exp(res.x)
47
48 print("\nFITTED PARAMETERS (deterministic fit, used for stochastic sims)")
49 print(f"beta    = {beta:.4f} per day")
50 print(f"k        = {k_fixed:.2f}")
51 print(f"alpha    = {alpha:.4f} per day")
52 print(f"gamma    = {gamma:.4f} per day")
53 print(f"delta    = {delta:.4f} per day")

```

```
54  
55 print(f"mean incubation = {1/alpha:.3f} days (1/alpha)")  
56 print(f"mean ill-in-bed = {1/gamma:.3f} days (1/gamma)")  
57 print(f"mean convalescent= {1/delta:.3f} days (1/delta)")
```

This script runs Monte carlo with Gillespie algorithm

```

1 rng = np.random.default_rng(1)
2
3 def gillespie_one(T=13.0):
4     S = int(round(N - I_obs[0]))
5     E = 0
6     I = int(round(I_obs[0]))
7     C = 0
8     R = 0
9     cum_ill = I
10
11     out_E = np.zeros_like(t_obs, dtype=float)
12     out_I = np.zeros_like(t_obs, dtype=float)
13     out_C = np.zeros_like(t_obs, dtype=float)
14
15     t = 0.0
16     idx = 0
17
18     # record at t=0
19     while idx < len(t_obs) and t_obs[idx] <= t:
20         out_E[idx], out_I[idx], out_C[idx] = E, I, C
21         idx += 1
22
23     while t < T:
24         rate_inf = beta * S * (E + k_fixed * I) / N
25         rate_EI = alpha * E
26         rate_IC = gamma * I
27         rate_CR = delta * C
28
29         rates = np.array([rate_inf, rate_EI, rate_IC, rate_CR], dtype=float)
30         a0 = rates.sum()
31         if a0 <= 0:
32             break
33         t += rng.exponential(1.0 / a0)
34
35         while idx < len(t_obs) and t_obs[idx] <= t:
36             out_E[idx], out_I[idx], out_C[idx] = E, I, C
37             idx += 1
38
39         r = rng.random() * a0
40         if r < rates[0]: # S -> E
41             if S > 0:
42                 S -= 1
43                 E += 1
44         elif r < rates[0] + rates[1]: # E -> I
45             if E > 0:
46                 E -= 1
47                 I += 1
48                 cum_ill += 1
49         elif r < rates[0] + rates[1] + rates[2]: # I -> C
50             if I > 0:
51                 I -= 1
52                 C += 1
53         else: # C -> R
54             if C > 0:
55                 C -= 1
56                 R += 1
57
58     while idx < len(t_obs):
59         out_E[idx], out_I[idx], out_C[idx] = E, I, C

```



```
60     idx += 1
61
62     return out_E, out_I, out_C, cum_ill
63
64 def summarize(arr):
65     mean = arr.mean(axis=0)
66     lo = np.quantile(arr, 0.025, axis=0)
67     hi = np.quantile(arr, 0.975, axis=0)
68     return mean, lo, hi
69
70 M = 2000
71 E_all = np.zeros((M, len(t_obs)))
72 I_all = np.zeros_like(E_all)
73 C_all = np.zeros_like(E_all)
74 cum_all = np.zeros(M)
75
76 for m in range(M):
77     E_all[m], I_all[m], C_all[m], cum_all[m] = gillespie_one()
78
79 I_mean, I_lo, I_hi = summarize(I_all)
80 C_mean, C_lo, C_hi = summarize(C_all)
81
82 Y14 = simulate_ode(beta, alpha, gamma, delta)
83 S14, E14, I14, C14, R14 = Y14
84
85 plt.figure()
86 plt.plot(days, I_obs, marker="o", linestyle="none", label="Observed I")
87 plt.plot(days, I14, label="Deterministic I")
88 plt.plot(days, C_obs, marker="s", linestyle="none", label="Observed C")
89 plt.plot(days, C14, label="Deterministic C")
90 plt.xlabel("Day of epidemic")
91 plt.ylabel("Number of pupils")
92 plt.title("Deterministic mean-field")
93 plt.legend()
94 plt.tight_layout()
95 plt.show()
```

Averaging results script

```

1 t_long = np.linspace(0, 80, 801)
2 I0 = float(I_obs[0])
3 y0 = [N - I0, 0.0, I0, 0.0, 0.0]
4 sol_long = solve_ivp(
5     seicr_rhs, (0, 80), y0, t_eval=t_long,
6     args=(beta, k_fixed, alpha, gamma, delta),
7     rtol=1e-8, atol=1e-10
8 )
9 S_l, E_l, I_l, C_l, R_l = sol_long.y
10
11 ever_infected_det = N - S_l[-1]
12 peak_EI_det = np.max(E_l + I_l)
13 day_peak_det = 1 + t_long[np.argmax(E_l + I_l)]
14
15 EI_all = E_all + I_all
16 peak_EI_stoch = EI_all.max(axis=1)
17 day_peak_stoch = 1 + t_obs[EI_all.argmax(axis=1)]
18
19 print("\nMODEL-BASED QUANTITIES")
20 print(f"(a) Total ever became ill (stochastic mean) = {cum_all.mean():.1f} pupils (
    compare: 512)")
21 print(f"(b) Mean infection->symptoms = 1/alpha = {1/alpha:.3f} days")
22 print(f"(c) Mean days ill in bed      = 1/gamma = {1/gamma:.3f} days")
23 print(f"    Mean extra convalescent    = 1/delta = {1/delta:.3f} days")
24 print(f"(d) Peak of (E+I) deterministic: day {day_peak_det:.2f}, size {peak_EI_det:.1
    f}")

```

Vaccination script

```
1 v_list = np.arange(0.1, 1.0, 0.1)
2 frac_unvacc_infected = []
3
4 for v in v_list:
5     V = v * N
6     S0v = (1 - v) * N - I0
7     if S0v < 0:
8         frac_unvacc_infected.append(np.nan)
9         continue
10
11     y0v = [S0v, 0.0, I0, 0.0, V]
12     solv = solve_ivp(
13         seicr_rhs, (0, 80), y0v, t_eval=[80],
14         args=(beta, k_fixed, alpha, gamma, delta),
15         rtol=1e-8, atol=1e-10
16     )
17     S_end = solv.y[0, -1]
18     infected_unvacc = (S0v - S_end) + I0
19     frac_unvacc_infected.append(infected_unvacc / ((1 - v) * N))
20
21 plt.figure()
22 plt.plot(v_list * 100, np.array(frac_unvacc_infected) * 100, marker="o")
23 plt.xlabel("Vaccinated before return (%)")
24 plt.ylabel("Infected among unvaccinated (%)")
25 plt.title("Vaccination impact predicted by the model")
26 plt.tight_layout()
27 plt.show()
```